Chapter 4 | Exercise solutions with Python

Self-Attention from Vectors to Matrices

This notebook works through Exercises 3 and 7–12. It begins with one query and three keys, then checks how batch and head axes, source masks, and permutations affect the computation. The final two exercises follow gradients through the attention calculation and its three input projections. Exercises 1, 2, 4, 5, and 6 are in the written solutions.

To rerun the notebook, open it inside a copy of `transformers-book` and use a Python kernel with NumPy from `requirements.txt`. Run the cells in order. The saved outputs show the results of running the complete notebook on a CPU.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    module = repo_root / "examples" / "ch04_attention" / "attention.py"
    if module.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")
```

3. Follow another query

Follow the second query, $\mathbf{q}_2 = (0, 1)$, through the unmasked example. Its first coordinate is zero, so only the second coordinate of each key contributes to the dot product. Each query and key has width $d_k = 2$. Divide each dot product by $\sqrt{2}$ before normalizing the row.

Key position	Key	Dot product with $(0, 1)$	Scaled score
1	$(1, 0)$	$0(1) + 1(0) = 0$	0
2	$(0, 1)$	$0(0) + 1(1) = 1$	$1/\sqrt{2}$
3	$(1, 1)$	$0(1) + 1(1) = 1$	$1/\sqrt{2}$

Let $r = \exp(1/\sqrt{2}) \approx 2.028114982$ and $D = 1 + 2r \approx 5.056229963$. The first exponential is $\exp(0) = 1$. The other two exponentials are both r. Softmax divides all three by D, giving the weight row

$$\left(\frac{1}{D}, \frac{r}{D}, \frac{r}{D}\right) \approx (0.197776,\ 0.401112,\ 0.401112).$$

Each value has two coordinates. Multiply both coordinates by the same weight from that key's position.

Position	Value	Exact contribution	Contribution to six decimals
1	$(1, 0)$	$(1/D, 0)$	$(0.197776, 0)$
2	$(0, 2)$	$(0, 2r/D)$	$(0, 0.802224)$
3	$(1, 1)$	$(r/D, r/D)$	$(0.401112, 0.401112)$

Add the first coordinates together and then the second coordinates together. Keeping the weights unrounded gives

$$\mathbf{o}_2 = \left(\frac{1+r}{D}, \frac{3r}{D}\right) \approx (0.598888,\ 1.203336).$$

The second coordinate exceeds one because it is a weighted average of value features $0, 2, 1$. The normalized quantities are the three attention weights.

Python uses zero-based indices, so `one_query_example(1)` selects query 2. The helper returns the scaled scores, weights, individual contributions, and output. Recover the raw dot products by multiplying the scaled scores by `sqrt(2)`. Then compare the row with the second row of the full matrix calculation.

```
from examples.ch04_attention.attention import (
    scaled_dot_product_attention,
)
from examples.ch04_attention.run_examples import (
    K, Q, V, one_query_example,
)

scores3, weights3, contributions3, output3 = one_query_example(1)
all_outputs3, all_weights3 = scaled_dot_product_attention(Q, K, V)
raw3 = scores3 * np.sqrt(Q.shape[-1])

exp_score3 = np.exp(1 / np.sqrt(2))
total3 = 1 + 2 * exp_score3
expected_weights3 = np.array([1, exp_score3, exp_score3]) / total3
expected_contributions3 = np.array([
    [1, 0], [0, 2 * exp_score3], [exp_score3, exp_score3],
]) / total3
```

Compare each returned array with the hand calculation. The final two checks compare the output and the weight row with the corresponding rows from the full matrix calculation.

```
np.testing.assert_allclose(raw3, [0, 1, 1])
np.testing.assert_allclose(scores3, [0, 1 / np.sqrt(2), 1 / np.sqrt(2)])
np.testing.assert_allclose(weights3, expected_weights3)
np.testing.assert_allclose(contributions3, expected_contributions3)
np.testing.assert_allclose(output3, expected_contributions3.sum(axis=0))
np.testing.assert_allclose(output3, all_outputs3[1])
np.testing.assert_allclose(weights3, all_weights3[1])

with np.printoptions(precision=6, suppress=True):
    print("Raw dot products:", raw3)
    print("Scaled scores:", scores3)
    print("Weights:", weights3)
    print("Contributions, one row per value:")
    print(contributions3)
    print("Output for query 2:", output3)
    print("Second row of the matrix output:", all_outputs3[1])
```

OUTPUT

```
Raw dot products: [0. 1. 1.]
Scaled scores: [0.       0.707107 0.707107]
Weights: [0.197776 0.401112 0.401112]
Contributions, one row per value:
[[0.197776 0.      ]
 [0.       0.802224]
 [0.401112 0.401112]]
Output for query 2: [0.598888 1.203336]
Second row of the matrix output: [0.598888 1.203336]
```

A dot product sums over **features**. For a fixed key *j*, it adds two coordinate products to make one score. Softmax then sums three exponentials over **key positions** for this one query. The read also sums over **positions**, adding one scaled value vector from each position. This last sum removes the position axis and retains the two value coordinates.

7. Follow the axes

Think of the batch and head axes as labels on a grid of matrices. Choose one example and one head. Its query matrix has three rows, its key matrix has five rows, and each query or key has eight features. A query needs one dot product with each of the five keys. Each dot product sums over eight paired features. Once the dot products are available, normalize over the five key positions and read their six-feature values.

Quantity	Shape	Axis meanings
Q	$(2, 4, 3, 8)$	batch, head, query position, query/key feature
K	$(2, 4, 5, 8)$	batch, head, key position, query/key feature
V	$(2, 4, 5, 6)$	batch, head, key position, value feature
Key transpose within each head	$(2, 4, 8, 5)$	batch, head, query/key feature, key position
Raw scores $QK^\top$	$(2, 4, 3, 5)$	batch, head, query position, key position
Scale $\sqrt{8}$	scalar	same scale at every position
Scaled scores S	$(2, 4, 3, 5)$	batch, head, query position, key position
Softmax row maxima and denominators	$(2, 4, 3, 1)$	one of each per query row
Shifted scores, exponentials, and weights A	$(2, 4, 3, 5)$	one entry per query and key
Output $O = AV$	$(2, 4, 3, 6)$	batch, head, query position, value feature

Within each slice, $(3, 8)(8, 5) \to (3, 5)$ contracts the feature width of eight. The softmax in the example uses `axis=-1` and `keepdims=True` for its reductions. The size-one axis lets NumPy subtract each row's maximum and divide by each row's denominator. The second product $(3, 5)(5, 6) \to (3, 6)$ contracts the five key and value positions. Each of the three query positions retains its own six-coordinate output.

```
from examples.ch04_attention.attention import (
    scaled_dot_product_attention,
    stable_softmax,
)
from examples.ch04_attention.run_examples import (
    Q as toy_q, K as toy_k, V as toy_v,
)

rng7 = np.random.default_rng(7)
q7 = rng7.normal(size=(2, 4, 3, 8))
k7 = rng7.normal(size=(2, 4, 5, 8))
v7 = rng7.normal(size=(2, 4, 5, 6))
keys_transposed7 = k7.swapaxes(-2, -1)
raw_scores7 = q7 @ keys_transposed7
scores7 = raw_scores7 / np.sqrt(q7.shape[-1])
row_max7 = scores7.max(axis=-1, keepdims=True)
exponentials7 = np.exp(scores7 - row_max7)
denominator7 = exponentials7.sum(axis=-1, keepdims=True)
output7, weights7 = scaled_dot_product_attention(q7, k7, v7)

assert keys_transposed7.shape == (2, 4, 8, 5)
assert raw_scores7.shape == scores7.shape == (2, 4, 3, 5)
assert row_max7.shape == denominator7.shape == (2, 4, 3, 1)
assert weights7.shape == (2, 4, 3, 5)
assert output7.shape == (2, 4, 3, 6)
np.testing.assert_allclose(weights7, exponentials7 / denominator7)
np.testing.assert_allclose(output7, weights7 @ v7)

slice_weights7 = stable_softmax(q7[1, 2] @ k7[1, 2].T / np.sqrt(8))
np.testing.assert_allclose(weights7[1, 2], slice_weights7)
np.testing.assert_allclose(output7[1, 2], slice_weights7 @ v7[1, 2])
print("One isolated batch/head slice agrees with the batched result.")
print("Weights shape:", weights7.shape)
print("Output shape:", output7.shape)
```

```
OUTPUT
One isolated batch/head slice agrees with the batched result.
Weights shape: (2, 4, 3, 5)
Output shape: (2, 4, 3, 6)
```

On a two-dimensional matrix, `.T` swaps the two matrix axes. On `k7`, `.T` reverses **all four** axes. The resulting last two axes are $(4, 2)$, which are the old head and batch axes. They cannot serve as the $(8, 5)$ feature and key axes of the matrix multiplication. The inner dimensions are now 8 and 4, and the leading dimensions $(2, 4)$ and $(8, 5)$ are also incompatible. `swapaxes(-2, -1)` keeps the batch and head labels in place.

```
print("Shape with swapaxes:", keys_transposed7.shape)
print("Shape with .T:", k7.T.shape)

try:
    q7 @ k7.T
except ValueError:
    print("ValueError: q7 @ k7.T cannot multiply these shapes.")
else:
    raise AssertionError("Expected the all-axis transpose to fail")
```

```
OUTPUT
Shape with swapaxes: (2, 4, 8, 5)
Shape with .T: (8, 5, 4, 2)
ValueError: q7 @ k7.T cannot multiply these shapes.
```

8. Combine padding and causality

Both examples store three queries, three keys, and three values. Example 1 has three real tokens. Example 2 has two real tokens and a right-padding slot at position 3. The vectors in that slot still contain the original toy numbers. Any change therefore comes from the mask.

Let C_{ij} indicate whether key j is at or before query i. Let R_{bj} indicate whether key j is a real token in example b. The allowed mask has entries $C_{ij} \wedge R_{bj}$. Insert singleton head and query axes into `key_valid` to give it shape $(2, 1, 1, 3)$. Boolean AND with the shared $(3, 3)$ lower triangle broadcasts that triangle across examples and repeats each example's key flags across query rows. The combined mask has shape $(2, 1, 3, 3)$, the same as the score array.

```
q8, k8, v8 = (
    np.broadcast_to(matrix, (2, 1, 3, 2))
    for matrix in (toy_q, toy_k, toy_v)
)
causal8 = np.tril(np.ones((3, 3), dtype=bool))
key_valid8 = np.array([[True, True, True], [True, True, False]])
key_flags8 = key_valid8[:, None, None, :]
allowed8 = causal8 & key_flags8
outputs8, weights8 = scaled_dot_product_attention(
    q8, k8, v8, allowed_mask=allowed8
)

assert key_flags8.shape == (2, 1, 1, 3)
assert allowed8.shape == weights8.shape == (2, 1, 3, 3)
assert outputs8.shape == (2, 1, 3, 2)
np.testing.assert_array_equal(allowed8[0, 0], causal8)
np.testing.assert_array_equal(weights8[~allowed8], 0.0)
np.testing.assert_array_equal(weights8[1, 0, :, 2], [0, 0, 0])
np.testing.assert_allclose(weights8.sum(axis=-1), 1)
np.testing.assert_allclose(weights8[1, 0, 2], [0.5, 0.5, 0])
np.testing.assert_allclose(outputs8[1, 0, 2], [0.5, 1])

for example in range(2):
    print(f"Example {example + 1} permissions (1 means allowed)")
    print(allowed8[example, 0].astype(int))
    print("Weights")
    print(np.round(weights8[example, 0], 4))
print("Example 2, padded query output:", outputs8[1, 0, 2])
```

OUTPUT

```
Example 1 permissions (1 means allowed)
[[1 0 0]
 [1 1 0]
 [1 1 1]]
Weights
[[1.     0.     0.    ]
 [0.3302 0.6698 0.    ]
 [0.2483 0.2483 0.5035]]
Example 2 permissions (1 means allowed)
[[1 0 0]
 [1 1 0]
 [1 1 0]]
Weights
[[1.     0.     0.    ]
 [0.3302 0.6698 0.    ]
 [0.5    0.5    0.    ]]
Example 2, padded query output: [0.5 1. ]
```

Look at the last row of the second example. Its query is $(1, 1)$, so its dot products with the two valid keys $(1, 0)$ and $(0, 1)$ are both 1. The allowed scaled scores tie. Softmax assigns the valid keys weight $1/2$ each and gives the padding key weight zero. The result is

$$\frac{1}{2}(1, 0) + \frac{1}{2}(0, 2) = (0.5, 1).$$

The third **column** of the mask governs the padding key. The third **row** is still a query over earlier real keys. A separate query-validity policy can discard its output or exclude the corresponding prediction from the loss. If we make the whole row false instead, there is no allowed key and no probability distribution with row sum one. The implementation raises a `ValueError` before doing the normalization.

```
empty_query_mask8 = allowed8.copy()
empty_query_mask8[1, 0, 2, :] = False

try:
    scaled_dot_product_attention(q8, k8, v8, empty_query_mask8)
except ValueError as error:
    assert "each query must allow at least one key" in str(error)
    print(f"ValueError: {error}")
else:
    raise AssertionError("Expected rejection of an all-false row")
```

```
OUTPUT
ValueError: each query must allow at least one key
```

9. Keep keys and values paired

Keep the queries where they are. Python indices `[2, 0, 1]` put the old third key and value first, the old first pair second, and the old second pair third. A score column travels with its key because the dot product depends on that key and the unchanged query. The row's softmax denominator uses the same three scores, so the corresponding weights travel with their columns. Each weight still multiplies the same value.

For example, the original first query adds $A_{11}v_1 + A_{12}v_2 + A_{13}v_3$. The new storage order adds $A_{13}v_3 + A_{11}v_1 + A_{12}v_2$. The summands are the same. The code checks both the rearranged weights and the unchanged outputs. The tolerance allows for small changes in addition order in floating-point arithmetic.

```
permutation9 = np.array([2, 0, 1])
keys9, values9 = toy_k[permutation9], toy_v[permutation9]
output9, weights9 = scaled_dot_product_attention(toy_q, toy_k, toy_v)
paired_output9, paired_weights9 = scaled_dot_product_attention(
    toy_q, keys9, values9
)
np.testing.assert_allclose(
    paired_weights9, weights9[:, permutation9], atol=1e-15
)
np.testing.assert_allclose(paired_output9, output9, atol=1e-15)
print("Original first weight row:", np.round(weights9[0], 4))
print("Reordered first weight row:", np.round(paired_weights9[0], 4))
print("Unmasked output, equal in both storage orders")
print(np.round(paired_output9, 4))
```

```
OUTPUT
Original first weight row: [0.4011 0.1978 0.4011]
Reordered first weight row: [0.4011 0.4011 0.1978]
Unmasked output, equal in both storage orders
[[0.8022 0.7967]
 [0.5989 1.2033]
 [0.7517 1.    ]]
```

With a causal mask, each allowed flag belongs to a particular query and key pair. Query 1 could originally read only key 1, which now occupies slot 2. Its new permission row must be `[False, True, False]`. Use the original mask's columns in the order `[2, 0, 1]`. The allowed scores and their paired values stay together, so the same weighted sum is computed.

```
causal9 = np.tril(np.ones((3, 3), dtype=bool))
causal_output9, causal_weights9 = scaled_dot_product_attention(
    toy_q, toy_k, toy_v, causal9
)
moved_mask9 = causal9[:, permutation9]
moved_output9, moved_weights9 = scaled_dot_product_attention(
    toy_q, keys9, values9, moved_mask9
)
np.testing.assert_allclose(
    moved_weights9, causal_weights9[:, permutation9], atol=1e-15
)
np.testing.assert_allclose(moved_output9, causal_output9, atol=1e-15)

print("Reordered causal permissions (1 means allowed)")
print(moved_mask9.astype(int))
print("Causal output, equal in both storage orders")
print(np.round(moved_output9, 4))
```

```
OUTPUT
Reordered causal permissions (1 means allowed)
[[0 1 0]
 [0 1 1]
 [1 1 1]]
Causal output, equal in both storage orders
[[1.     0.    ]
 [0.3302 1.3395]
 [0.7517 1.    ]]
```

A fresh lower triangle uses the **new slot numbers** to decide what is available. In the first row it allows slot 1, which stores the old third value $(1, 1)$. The original causal read returns the old first value $(1, 0)$.

Reordering only the keys changes which values the weights multiply even without a mask. For the first query, the new weights are $[A_{13}, A_{11}, A_{12}]$. With values left in their old order, the result is $A_{13}v_1 + A_{11}v_2 + A_{12}v_3$. The second value now receives the old first key's weight. The numerical check below uses the same toy inputs in both counterexamples.

```
wrong_causal9, _ = scaled_dot_product_attention(
    toy_q, keys9, values9, causal9
)
keys_only9, keys_only_weights9 = scaled_dot_product_attention(
    toy_q, keys9, toy_v
)

np.testing.assert_allclose(wrong_causal9[0], toy_v[2])
assert not np.allclose(wrong_causal9[0], causal_output9[0])
np.testing.assert_allclose(keys_only_weights9, paired_weights9)
assert not np.allclose(keys_only9[0], output9[0])

print("First causal output with original permissions:", causal_output9[0])
print("First causal output with a fresh triangle:", wrong_causal9[0])
print("First unmasked output:", np.round(output9[0], 4))
print("First unmasked output after moving only K:",
      np.round(keys_only9[0], 4))
```

```
OUTPUT
First causal output with original permissions: [1. 0.]
First causal output with a fresh triangle: [1. 1.]
First unmasked output: [0.8022 0.7967]
First unmasked output after moving only K: [0.5989 1.    ]
```

10. Separate order from parallel computation

Start from the three input rows H and place them in order `[2, 0, 1]` **before** forming queries, keys, and values. Each projection applies the same matrix to every row, so the new Q, K, and V all inherit the same permutation. No positional feature is added and all query–key pairs are allowed.

As in Exercise 9, moving the key and value pairs keeps a query's weighted sum unchanged. This time the query rows move too, so the output for the old third query appears first. If $\pi(i)$ maps a new slot to its old index, the scores at new slots (i, j) are the old scores at $(\pi(i), \pi(j))$. Softmax moves the weights in the same way, and the value sum gives the old output at $\pi(i)$. The new weight matrix is the old matrix indexed by the permutation on both axes. The new output is the old output indexed only on its query axis.

```
from examples.ch04_attention.run_examples import H, W_Q, W_K, W_V

permutation10 = np.array([2, 0, 1])
h_permuted10 = H[permutation10]
q10 = h_permuted10 @ W_Q
k10 = h_permuted10 @ W_K
v10 = h_permuted10 @ W_V
output10, weights10 = scaled_dot_product_attention(toy_q, toy_k, toy_v)
new_output10, new_weights10 = scaled_dot_product_attention(q10, k10, v10)

np.testing.assert_allclose(q10, toy_q[permutation10])
np.testing.assert_allclose(k10, toy_k[permutation10])
np.testing.assert_allclose(v10, toy_v[permutation10])
np.testing.assert_allclose(
    new_weights10, weights10[np.ix_(permutation10, permutation10)]
)
np.testing.assert_allclose(new_output10, output10[permutation10])

print("Original outputs, in old row order")
print(np.round(output10, 4))
print("New outputs, in old row order [3, 1, 2]")
print(np.round(new_output10, 4))
```

OUTPUT

```
Original outputs, in old row order
[[0.8022 0.7967]
 [0.5989 1.2033]
 [0.7517 1.    ]]
New outputs, in old row order [3, 1, 2]
[[0.7517 1.    ]
 [0.8022 0.7967]
 [0.5989 1.2033]]
```

This property is called **permutation equivariance**. Moving an input to a different row moves its output with it. Under these conditions, the calculation uses a token's projected features and the collection of projected keys and values. The row index organizes storage and contributes no absolute-position signal. Adding positional information or fixing a causal mask to the chronological order changes the conditions of this check.

There is a separate question about which calculations depend on each other. Within one attention layer, once its input rows are available, all of that layer's queries, keys, and values can be projected together. Each query's output reads those same input projections. It does not use any other query's output from that layer. Matrix products and row-wise softmax can therefore evaluate the query rows together. This still holds for a known sequence with a causal mask because the mask supplies each row's permissions before the calculation.

The next layer takes the preceding layer's outputs as input, so that dependency is between layers. Autoregressive generation has an additional dependency between time steps. The next token selected at the current step becomes part of the prefix used at the following step. The new token's input is available only after that selection.

11. Choose a finite-difference step (advanced)

Imagine increasing a single entry of the query matrix Q by a tiny amount. The change propagates through dot products, softmax, and the weighted value sum. We want to check the derivative of one scalar loss with respect to that entry. The same procedure works for entries of the key and value matrices K and V.

Let O be the attention output and G the demonstration's fixed incoming gradient. Use the scalar loss

$$L = \langle O, G \rangle = \sum_{i,j} O_{ij} G_{ij}.$$

This choice gives $\partial L / \partial O = G$. To probe a coordinate θ in Q, K, or V, hold everything else fixed and evaluate the same loss twice. Its central-difference derivative is

$$D_\varepsilon L(\theta) = \frac{L(\theta + \varepsilon) - L(\theta - \varepsilon)}{2\varepsilon}.$$

The mask stays fixed in both evaluations. We check every coordinate of the three independent inputs and take the maximum absolute difference from the manual backward pass.

```
from examples.ch04_attention import run_examples as demo
from examples.ch04_attention.attention import (
    attention_backward,
    scaled_dot_product_attention,
)

causal = np.tril(np.ones((3, 3), dtype=bool))
incoming = np.array(
    [[0.2, -0.4], [0.5, 0.3], [-0.1, 0.6]], dtype=np.float64
)
qkv = (demo.Q, demo.K, demo.V)
assert all(x.dtype == np.float64 for x in qkv)
print("Fixed causal mask, 1 means allowed:\n", causal.astype(int))
print("Fixed incoming gradient G:\n", incoming)
```

```
OUTPUT
Fixed causal mask, 1 means allowed:
 [[1 0 0]
 [1 1 0]
 [1 1 1]]
Fixed incoming gradient G:
 [[ 0.2 -0.4]
 [ 0.5  0.3]
 [-0.1  0.6]]
```

Give the helper `step=1e-6` in its signature and remove the line that assigned `step` inside it. The default reproduces the demonstration. Calling it with a different step changes the size of the two perturbations. Copies protect the original matrices, and resetting the selected coordinate prevents one test from affecting the next.

The code contracts the output difference with G before dividing by 2ε. Because G is fixed, $\langle O_+ - O_-, G \rangle = L_+ - L_-$. This is the same central difference in exact arithmetic.

```
def gradient_max_error(q, k, v, upstream, allowed_mask, step=1e-6):
    # Check attention gradients with a configurable step.
    analytic = attention_backward(q, k, v, upstream, allowed_mask)
    inputs = [q.copy(), k.copy(), v.copy()]
    max_error = 0.0

    for array, expected in zip(inputs, analytic):
        for index in np.ndindex(array.shape):
            original = array[index]
            array[index] = original + step
            plus, _ = scaled_dot_product_attention(
                *inputs, allowed_mask
            )
            array[index] = original - step
            minus, _ = scaled_dot_product_attention(
                *inputs, allowed_mask
            )
            array[index] = original
            numerical = np.sum((plus - minus) * upstream) / (2 * step)
            error = abs(numerical - expected[index])
            max_error = max(max_error, error)

    return max_error
```

```
steps = (1e-2, 1e-4, 1e-6, 1e-8)
print("     step   maximum absolute error")
for step in steps:
    error = gradient_max_error(*qkv, incoming, causal, step=step)
    print(f"{step:9.0e}   {error:.3e}")

np.testing.assert_allclose(
    gradient_max_error(*qkv, incoming, causal),
    demo.gradient_max_error(*qkv, incoming, causal),
)
```

```
OUTPUT
     step   maximum absolute error
    1e-02   1.609e-07
    1e-04   1.595e-11
    1e-06   5.682e-11
    1e-08   7.228e-09
```

At 10^{-2} the error is about 1.61×10^{-7}. Reducing the step to 10^{-4} brings it down to about 1.60×10^{-11}. In exact arithmetic, the two Taylor expansions cancel their even-order terms when subtracted. After dividing by 2ε, the first remaining approximation error is proportional to ε^2 for a smooth function. A large step therefore measures the slope across too wide an interval.

At 10^{-6} the measured error grows slightly, and at 10^{-8} it reaches about 7.23×10^{-9}. The two forward evaluations become almost equal as the step shrinks. Subtracting them discards many shared significant digits, and dividing by a tiny 2ε amplifies the remaining rounding error. The smallest observed error in this float64 sweep comes at 10^{-4}. The balance depends on the scale and curvature of the function and on arithmetic precision.

12. Learn through the projections (advanced)

In Exercise 11 we could move an entry of Q without moving K or V. Now each one is produced from the same three input rows in H. Changing an entry of H can change all three projections, so the derivative with respect to that entry must collect three contributions. Changing a weight in one projection changes only that projection.

Make a copy of the teaching key projection weight matrix W^K and set its first entry to 0.5. Keep the query and key projection weights as separate variables, including during the numerical checks. The two weight matrices have the same shape. Each acts in its own projection, and their values now differ. Keep the causal mask and the incoming gradient G from Exercise 11. The scalar loss is still $\langle O, G \rangle$.

```
hidden = demo.H.copy()
w_query = demo.W_Q.copy()
w_key = demo.W_K.copy()
w_value = demo.W_V.copy()
w_key[0, 0] = 0.5

q_proj = hidden @ w_query
k_proj = hidden @ w_key
v_proj = hidden @ w_value
projected_output, projected_weights = scaled_dot_product_attention(
    q_proj, k_proj, v_proj, causal
)
with np.printoptions(precision=6, suppress=True):
    print("Revised K:\n", k_proj)
    print("Causal attention weights:\n", projected_weights)
    print("Output O:\n", projected_output)
print(f"Scalar loss = {np.sum(projected_output * incoming):.12f}")
```

OUTPUT

```
Revised K:
 [[0.5 0. ]
 [0.  1. ]
 [0.5 1. ]]
Causal attention weights:
 [[1.       0.       0.      ]
 [0.330238 0.669762 0.      ]
 [0.224606 0.319866 0.455527]]
Output O:
 [[1.       0.      ]
 [0.330238 1.339523]
 [0.680134 1.09526 ]]
Scalar loss = 1.356118664858
```

Write $\overline{X}$ for the derivative $\partial L/\partial X$, an array with the same shape as X. First use the manual attention backward pass to obtain $\overline{Q}$, $\overline{K}$, and $\overline{V}$. These are the three incoming gradients for the projections.

To derive the query projection's gradients, look at coordinate (i, b) of its output. The row index i selects a token, a runs over the four input features, and b selects one of the two query features.

$$Q_{ib} = \sum_a H_{ia} W^Q_{ab}.$$

Changing weight W^Q_{ab} changes Q_{ib} by H_{ia} times the weight change, for every token i. Multiply each contribution by $\overline{Q}_{ib}$ and add across tokens. Changing input feature H_{ia} changes every query coordinate Q_{ib} of that token by W^Q_{ab} times the input change. Multiply by $\overline{Q}_{ib}$ and add across query features. The two sums give

$$\begin{aligned} \overline{W}^Q_{ab} &= \textstyle\sum_i H_{ia}\overline{Q}_{ib}, & \overline{W}^Q &= H^\mathsf{T}\overline{Q}, \\ (\overline{H}_Q)_{ia} &= \textstyle\sum_b \overline{Q}_{ib} W^Q_{ab}, & \overline{H}_Q &= \overline{Q}(W^Q)^\mathsf{T}. \end{aligned}$$

The index being summed tells you which axis the matrix product contracts. The weight derivative sums over token rows, whereas the input derivative sums over the output features of that projection.

The same product rule applies to $K = HW^K$ and $V = HW^V$. The projection weight gradients are

$$\overline{W}^K = H^\mathsf{T}\overline{K}, \qquad \overline{W}^V = H^\mathsf{T}\overline{V}.$$

A perturbation to H enters the loss through the queries, the keys, and the values. The first-order changes along these paths add, giving

$$\overline{H} = \overline{Q}(W^Q)^\mathsf{T} + \overline{K}(W^K)^\mathsf{T} + \overline{V}(W^V)^\mathsf{T}.$$

Each weight gradient is a product of a $(4, 3)$ matrix and a $(3, 2)$ matrix, so it has shape $(4, 2)$. Each term for $\overline{H}$ is a $(3, 2)$ matrix times a $(2, 4)$ matrix, so the sum has shape $(3, 4)$. Those shapes agree with the shapes of the four inputs.

```
grad_q, grad_k, grad_v = attention_backward(
    q_proj, k_proj, v_proj, incoming, causal
)
grad_w_query = hidden.T @ grad_q
grad_w_key = hidden.T @ grad_k
grad_w_value = hidden.T @ grad_v

through_query = grad_q @ w_query.T
through_key = grad_k @ w_key.T
through_value = grad_v @ w_value.T
grad_hidden = through_query + through_key + through_value

analytic = (grad_hidden, grad_w_query, grad_w_key, grad_w_value)
parameters = (hidden, w_query, w_key, w_value)
names = ("H", "W_Q", "W_K", "W_V")
with np.printoptions(precision=8, suppress=True):
    for name, value, gradient in zip(names, parameters, analytic):
        assert gradient.shape == value.shape
        print(f"dL/d{name} {gradient.shape}\n{gradient}\n")
```

OUTPUT

```
dL/dH (3, 4)
[[-0.05472504 -0.12508993  0.34265859 -0.16616466]
 [ 0.0612618   0.16944318  0.30289416  0.39284816]
 [-0.08343843  0.08073669 -0.04555275  0.27331649]]

dL/dW_Q (4, 2)
[[-0.06908173  0.10945007]
 [-0.07690166  0.12508993]
 [-0.06908173  0.10945007]
 [-0.08472159  0.14072979]]

dL/dW_K (4, 2)
[[-0.13816346 -0.15380332]
 [ 0.10945007  0.12508993]
 [-0.13816346 -0.15380332]
 [ 0.24761354  0.27889326]]

dL/dW_V (4, 2)
[[0.29710584 0.10715184]
 [0.25734141 0.66616466]
 [0.29710584 0.10715184]
 [0.56023557 1.05901282]]
```

For example, consider the first feature of the first input token. The first query can read only the first key under the causal mask. Its one attention weight is always one, so its query gradient is zero. The first key can affect later queries, and its first gradient coordinate is about -0.10945007. The first input feature reaches that key coordinate through the revised weight 0.5. It has no direct path into the value projection because that row of W^V is zero. This gives $\overline{H}_{11} = 0 + 0.5(-0.10945007) + 0 \approx -0.05472504$, the upper-left entry of the printed gradient.

To check the derivation, rerun the full forward calculation after changing one coordinate of one of the four independent arrays. The numerical helper uses only the scalar loss function, so it can catch a mistake in the analytic projection derivatives. In particular, changing H recomputes all three projections, while changing W^Q leaves W^K fixed and vice versa.

```
def projection_loss(h, w_q, w_k, w_v):
    output, _ = scaled_dot_product_attention(
        h @ w_q, h @ w_k, h @ w_v, causal
    )
    return np.sum(output * incoming)


def numerical_projection_gradients(parameters, step=1e-6):
    # Perturb each input entry and rerun the whole forward pass.
    copied = [parameter.copy() for parameter in parameters]
    gradients = []
    for array in copied:
        numerical = np.empty_like(array)
        for index in np.ndindex(array.shape):
            original = array[index]
            array[index] = original + step
            loss_plus = projection_loss(*copied)
            array[index] = original - step
            loss_minus = projection_loss(*copied)
            array[index] = original
            numerical[index] = (loss_plus - loss_minus) / (2 * step)
        gradients.append(numerical)
    return gradients
```

```
numerical = numerical_projection_gradients(parameters)
print("Input   maximum absolute error")
for name, exact, estimate in zip(names, analytic, numerical):
    max_error = np.max(np.abs(exact - estimate))
    print(f"{name:<5}   {max_error:.3e}")
    np.testing.assert_allclose(exact, estimate, atol=1e-8, rtol=0)

assert w_query[0, 0] == 1.0 and w_key[0, 0] == 0.5
assert not np.shares_memory(w_query, w_key)
```

```
OUTPUT
Input   maximum absolute error
H       1.752e-10
W_Q     1.287e-10
W_K     1.368e-10
W_V     1.056e-10
```

All four comparisons pass the 10^{-8} absolute tolerance in the saved float64 run. The key and query weight gradients differ. Each comes from its own projection, and the numerical test perturbs just one of the two matrices at a time. The match for $\overline{H}$ checks the sum of the three paths against a single perturbation that reaches all of them at once.